



1  
**XIAMEN**  
UNIVERSITY

# DIGITAL LOGIC

## Combinational Logic

### Chapter 3.2

**Dr. Fei Chao**

**Associate Professor**

**Cognitive Science Department**

**School of Information Science and Technology**

**Xiamen University**

**fchao@xmu.edu.cn**

## Several practical problems in the combinational logic design

### I. Combinational logic circuit design with “Don’t care” conditions

In some practical problems, due to the existence of mutual constraints between the function outputs and variables or a particular limitation of the problem, resulting as the logic function has nothing to do with several input values, usually this type of problem categorized as the “don’t care” logic problem; Logical functions that describe such problems are called “don’t care term” logical functions.

**Concept of the “don’t care Minterm”:** due to the existence of mutual constraints between the function outputs and variables or a particular limitation of the problem, some combinations of input values cannot effect the output, the corresponding Minterms of these combinations are called don’t care Minterms,.



**When the “Sum of Minterm” expression is used to describe a logical problem that contains don’t care conditions, whether the function expression contains a don’t care item and whether the don’t care term is 1 or 0 does not affect the actual logic of the function.**

**Note that: in this case, when simplifying such logic functions, the randomness of don’t care terms can often make the logic functions better simplified, so that the designed circuit can be simplified further!**

**Example:** A combinational logic circuit is designed to determine whether a 1-bit decimal number represented by exceed-3 codes is a composite number.

**Solution:** assume the input variables are **ABCD**, output is **F**, when the decimal numbers represented by ABCD are (4, 6, 8, 9) , F is 1, otherwise F is 0.

Due to the encoding rule of exceed-3 code, the combinations of **ABCD** cannot be 0000, 0001, 0010, 1101, 1110, 1111; therefore, this question can be categorized as the don't care condition problem.

In this case, the six input combinations are the don't care minterms, i.e., the function input of these 6 minterms can be assigned casually to 0 or 1, usually, marked as “d”.

According to the analysis, the truth table describing the problem can be established as shown in the following table.

| A | B | C | D | F |  | A | B | C | D | F |
|---|---|---|---|---|--|---|---|---|---|---|
| 0 | 0 | 0 | 0 | d |  | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | d |  | 1 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | d |  | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |  | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 |  | 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 |  | 1 | 1 | 0 | 1 | d |
| 0 | 1 | 1 | 0 | 0 |  | 1 | 1 | 1 | 0 | d |
| 0 | 1 | 1 | 1 | 1 |  | 1 | 1 | 1 | 1 | d |

The logic function express can be obtained:

$$F(A,B,C,D) = \sum m(7,9,11,12) + \sum d(0,1,2,13,14,15)$$

➡ If do not consider don't care items, the Karnaugh map of function F is shown in the figure below.

| AB \ CD |    | AB |    |    |    |
|---------|----|----|----|----|----|
|         |    | 00 | 01 | 11 | 10 |
| CD      | 00 | d  | 0  | 1  | 0  |
|         | 01 | d  | 0  | d  | 1  |
|         | 11 | 0  | 1  | d  | 1  |
|         | 10 | d  | 0  | d  | 0  |

Combine the 1 value squares, after the simplification, the logic function is:

$$F(A, B, C, D) = \bar{A}\bar{B}D + A\bar{B}\bar{C} \cdot \bar{D} + \bar{A}BCD$$

➡ If consider the don't care items, the Karnaugh map of function F is shown in the figure below.

|    |    | AB |    |    |    |
|----|----|----|----|----|----|
|    |    | 00 | 01 | 11 | 10 |
| CD | 00 | d  | 0  | 1  | 0  |
|    | 01 | d  | 0  | d  | 1  |
|    | 11 | 0  | 1  | d  | 1  |
|    | 10 | d  | 0  | d  | 0  |

We can deal with the don't care terms d(13, 14, 15) as 1 value square, and regard d(0,1,2) as 0 value square, then, we can have the simplified form:

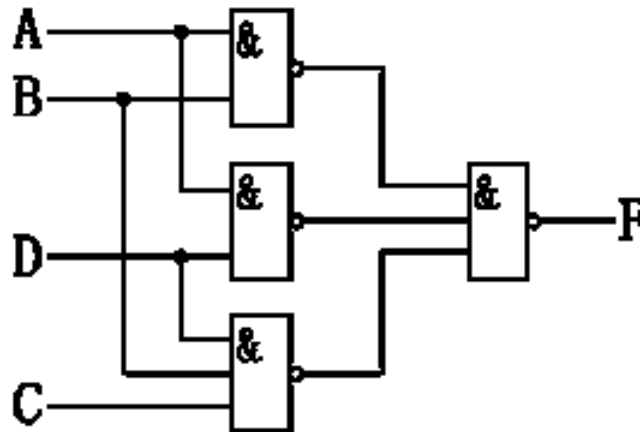
$$F(A,B,C,D) = AB + AD + BCD$$

Obviously, the latter expression is simpler than the previous expression.

**Assume that we must use NAND gates to implement the circuit for the given logic function. The simplest expression of F can be transformed into an AND-Non-AND expression:**

$$F(A, B, C, D) = \overline{\overline{AB + AD + BCD}} = \overline{\overline{AB} \cdot \overline{AD} \cdot \overline{BCD}}$$

**The corresponding logic circuit is illustrated as:**



It can be seen that, when designing a combinational logic circuit that includes don't care terms, properly don't care terms are used to simplify functions, and usually the designed circuit can be made simpler.



## II. Multiple-output combinational logic circuit design

In practical problems, there are a large number of problems that generate multiple output functions from the same set of input variables. The combined logic circuit that implements such problems is called a combinational logic circuit of multiple output functions.

The key to achieve the simplest combinational circuit is to find the common items of each output function in the function simplification, so as to realize the “sharing” of logic gates in the logic circuit, so as to achieve the simplest overall circuit structure.

## Example: Design a Full Adder (Optional Logic Gate)

### Solution:

Full adder: It is able to add two <sup>1,2</sup> 1-bit binary numbers and  
3. "carry" from the lower bits to generate a logical sum for the  
1. "sum" bit and the "carry" for high bit. The full adder can be  
used to add two n-bit numbers.

Obviously, the full adder has 3 input variables and 2 output functions.

Assume: the summand, addend, and carry from the low bit are represented by  $\underline{A_i}$ ,  $\underline{B_i}$  and  $\underline{C_{i-1}}$ , the sum and carry are represented by  $\underline{S_i}$  and  $\underline{C_i}$ .

According to the binary addition algorithm, the full adder's truth table can be listed as shown in the following table.

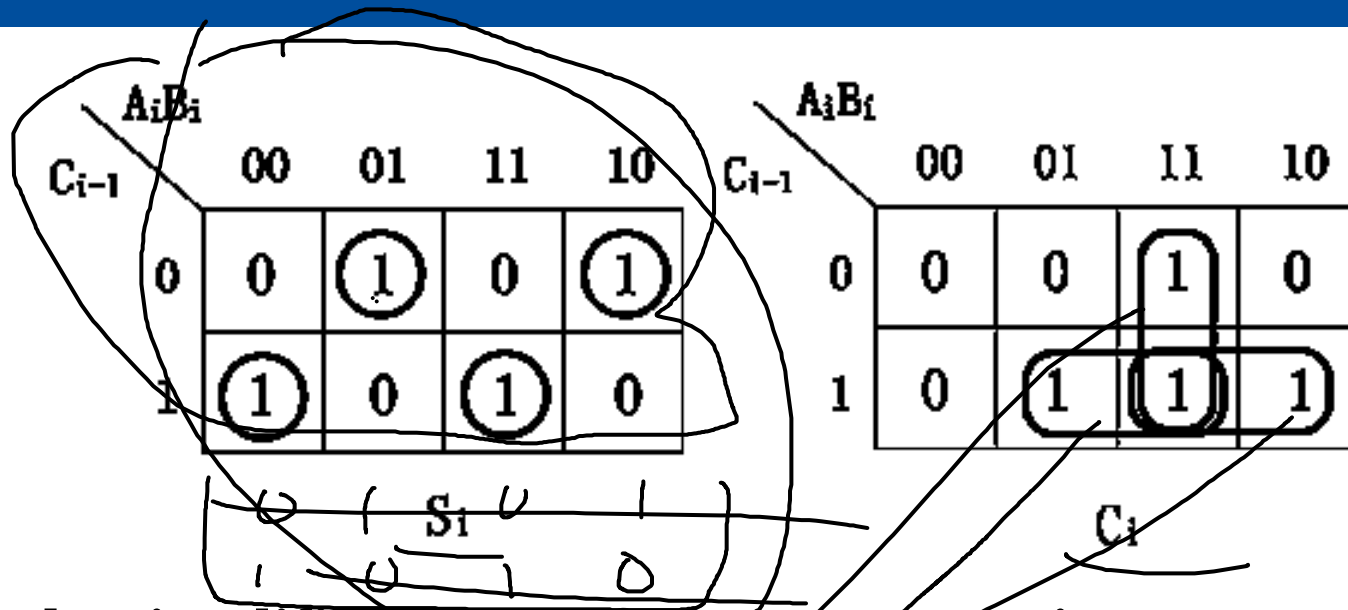
| $\underline{A_i}$ | $\underline{B_i}$ | $\underline{C_{i-1}}$ | $\underline{S_i}$ | $\underline{C_i}$ |
|-------------------|-------------------|-----------------------|-------------------|-------------------|
| 0                 | 0                 | 0                     | 0                 | 0                 |
| 0                 | 0                 | 1                     | 1                 | 0                 |
| 0                 | 1                 | 0                     | 1                 | 0                 |
| 0                 | 1                 | 1                     | 0                 | 1                 |
| 1                 | 0                 | 0                     | 1                 | 0                 |
| 1                 | 0                 | 1                     | 0                 | 1                 |
| 1                 | 1                 | 0                     | 0                 | 1                 |
| 1                 | 1                 | 1                     | 1                 | 1                 |

The output function expression can be written from the truth table:

$$\underline{S_i(A_i, B_i, C_{i-1})} = \sum m(\underline{1, 2, 4, 7})$$

$$\underline{C_i(A_i, B_i, C_{i-1})} = \sum m(\underline{3, 5, 6, 7})$$

Assuming that the above functions are simplified using the Karnaugh map, the two corresponding Karnaugh maps can be made as shown in the following figure.



The simplified output function expressions are

$$S_i = \overline{A_i} \cdot \overline{B_i} C_{i-1} + \overline{A_i} B_i \overline{C_{i-1}} + A_i \overline{B_i} \cdot \overline{C_{i-1}} + A_i B_i C_{i-1}$$

$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1}$$

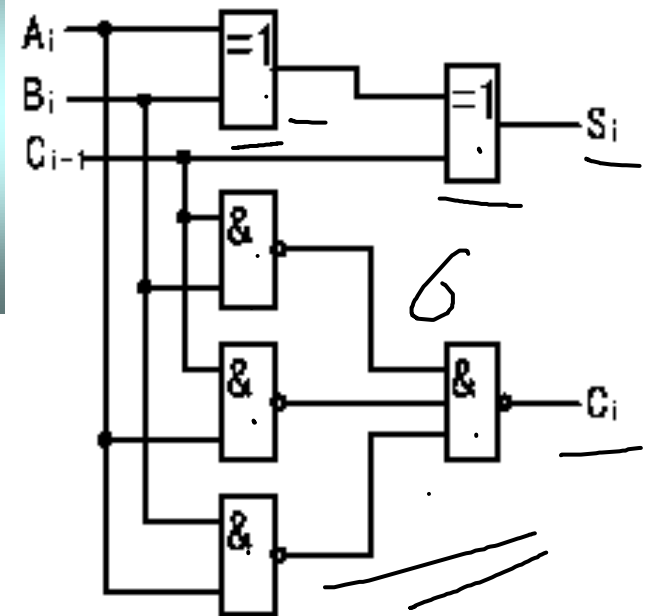
where,  $S_i$  is the canonical “AND-OR” term, it is also the simplest “AND-OR” term.

When use XOR gates and NAND gates to implement a circuit for the given function, the expressions can be transformed as follows:

$$\begin{aligned}
 S_i &= \overline{A_i} \cdot \overline{B_i} C_{i-1} + \overline{A_i} B_i \overline{C_{i-1}} + A_i \overline{B_i} \overline{C_{i-1}} + A_i B_i C_{i-1} \\
 &= \overline{A_i} (\overline{B_i} C_{i-1} + B_i \overline{C_{i-1}}) + A_i (\overline{B_i} \overline{C_{i-1}} + B_i C_{i-1}) \\
 &= \overline{A_i} (B_i \oplus C_{i-1}) + A_i (B_i \oplus C_{i-1}) \\
 &= A_i \oplus (B_i \oplus C_{i-1})
 \end{aligned}$$

$\overline{\overline{X}} = X$

$$\begin{aligned}
 C_i &= A_i B_i + A_i C_{i-1} + B_i C_{i-1} \\
 &= \overline{\overline{A_i B_i} \cdot \overline{A_i C_{i-1}} \cdot \overline{B_i C_{i-1}}}
 \end{aligned}$$



The corresponding logic circuit diagram is shown on the right.

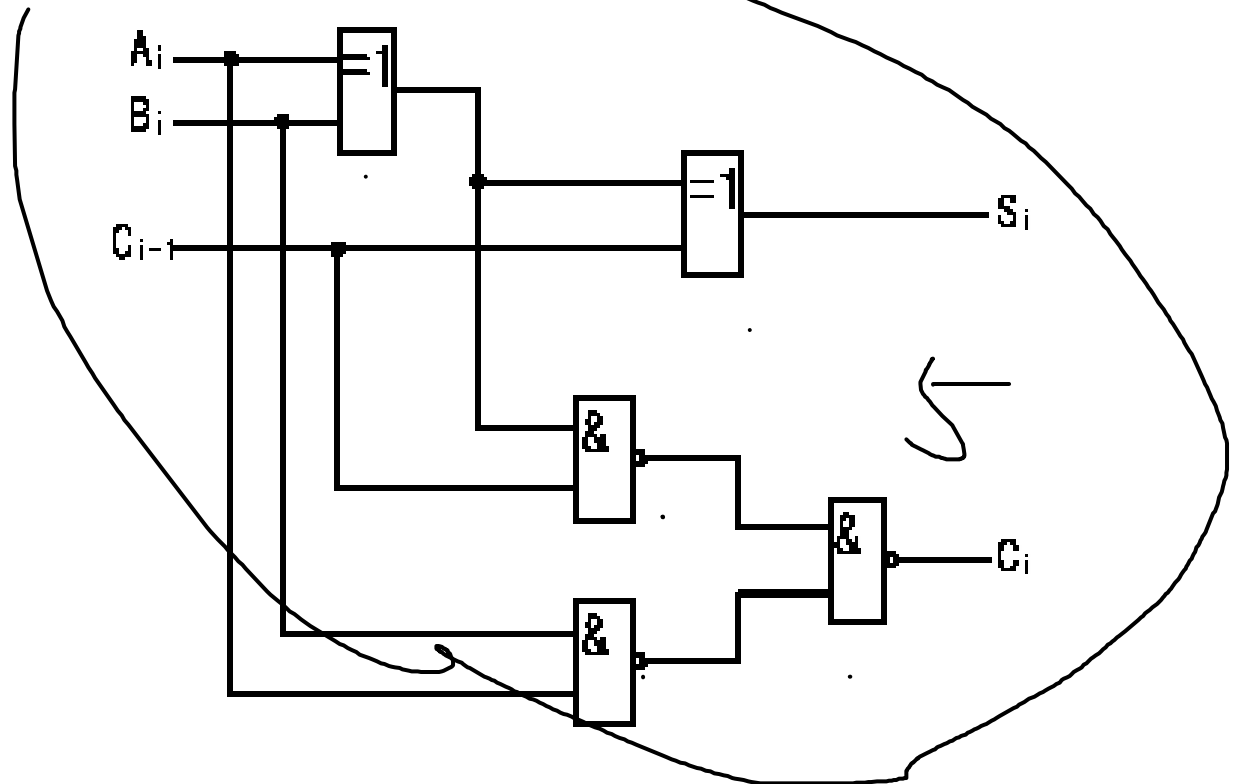
For each single output circuit, both  $A_i$  and  $C_i$  are the simplest, however, from the perspective view of the entire design, the circuit is not the simplest.

When designing by this multi-output function combination circuit, we can transform  $C_i$  to the following form:

$$\begin{aligned}
 C_i &= \overline{A_i} B_i C_{i-1} + A_i \overline{B_i} C_{i-1} + A_i B_i \overline{C_{i-1}} + A_i B_i C_{i-1} \\
 &= (\overline{A_i} B_i + A_i \overline{B_i}) C_{i-1} + A_i B_i (\overline{C_{i-1}} + C_{i-1}) \\
 &= (A_i \oplus B_i) C_{i-1} + A_i B_i \quad \leftarrow \overline{A_i B_i} \\
 &= \overline{(A_i \oplus B_i) C_{i-1}} \cdot A_i B_i
 \end{aligned}$$

After transformation,  $S_i$  ( $S_i = A_i \oplus B_i \oplus C_{i-1}$ ) and  $C_i$  have a share term  $A_i \oplus B_i$ .

After the transformation, the circuits can be combined to make them share the same XOR gate, so that the entire design is further simplified. The logic circuit diagram is shown in the figure below.



### III, the combination of logic circuit design without complement-variables

In the design of certain problems, in order to reduce the connections between components, only the true variable is provided at the input of the logic circuit, and no complement variable is provided.

When designing such a circuit, if the original variable is directly converted into the corresponding complement variable, the processing result is often uneconomical. Therefore, several appropriate transformations are usually performed, so as to reduce the number of not-gates as much as possible.



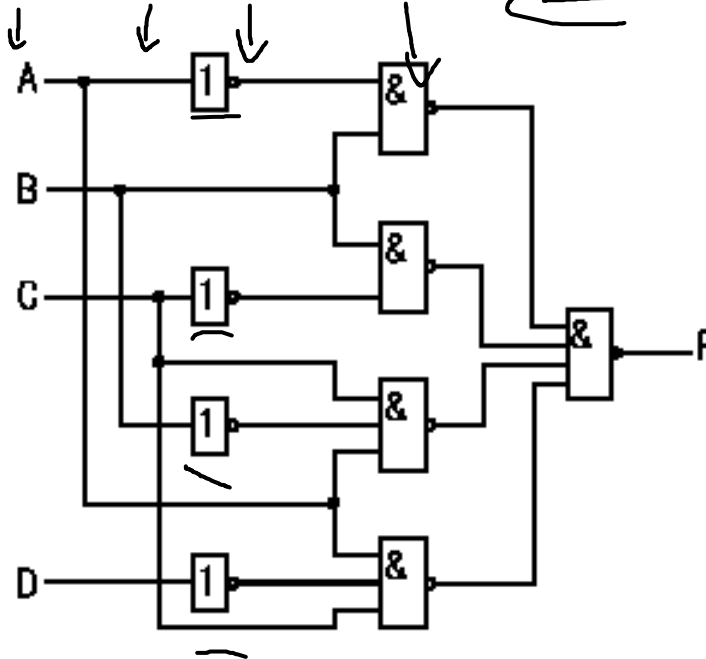
**Example,** When the input does not provide complement variables, use the NAND gate to implement the following logic function

$$F(A, B, C, D) = AB + BC + ABC + ACD$$

**Solution:** Since the given function is already the simplest AND-OR expression, it can be directly transformed into NAND expression.

$$F(A, B, C, D) = \overline{\overline{AB} + \overline{BC} + \overline{ABC} + \overline{ACD}}$$

$$= \overline{\overline{AB}} \cdot \overline{\overline{BC}} \cdot \overline{\overline{ABC}} \cdot \overline{\overline{ACD}}$$



The corresponding logic circuit is shown in the right figure.

If the expression of function F is organized as follows,

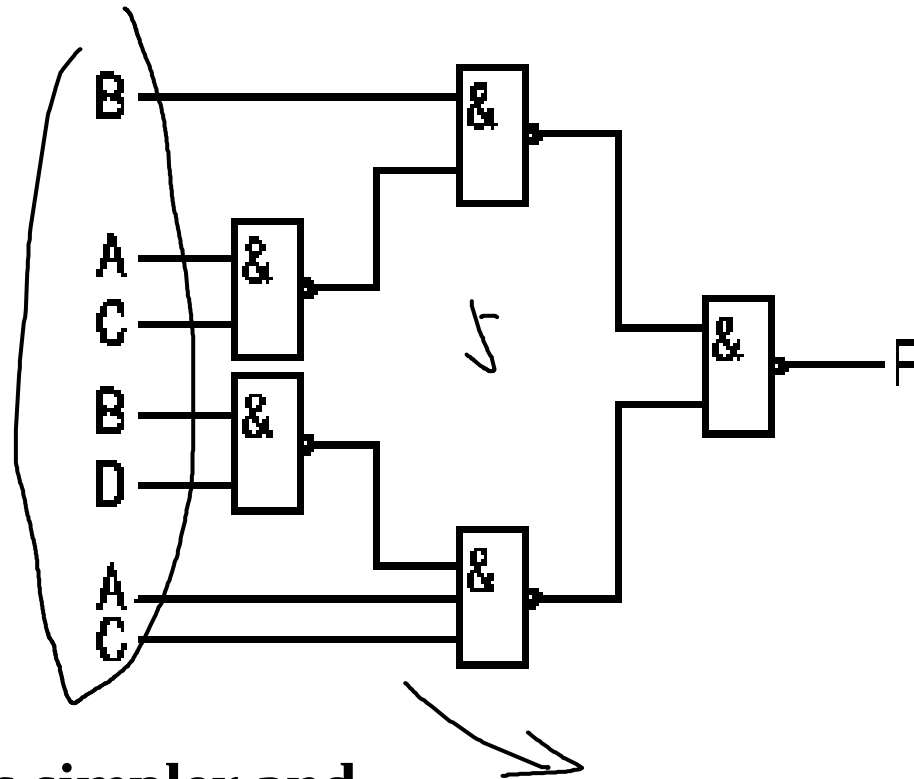
$$F(A, B, C, D) = \bar{A}B + B\bar{C} + \underline{\bar{A}\bar{B}\bar{C}} + AC\bar{D}$$

$$= B(\bar{A} + \bar{C}) + AC(\bar{B} + \bar{D})$$

$$= \underline{\underline{BAC}} + \underline{\underline{ACBD}}$$

$$= \underline{\underline{BAC}} \cdot \underline{\underline{ACBD}}$$

The corresponding logic circuit is shown in the right figure.



Obviously, this diagram is simpler and more reasonable than the previous one.

## 4.4 Hazard in combinational logic circuits

Since the signal passes through any logic gates and wires will produce a time delay, the output does not immediately reach a steady state when all inputs of the circuit reach a steady state.

### 4.4.1 Emergence of race and hazard

The length of the delay time in each path of the logic circuit is related to the number of stages the signal passes through, and is related to the delay time of the specific logic gate, and also related to the length of the wire.

Therefore, the input signal passes through different paths to the output terminal with various durations. This phenomenon is called **the race phenomenon**.

**Race:** Due to the influence of the delay time, the input signal passes through different paths and arrives the output end at different time. This phenomenon is called race.

**Types of race:**

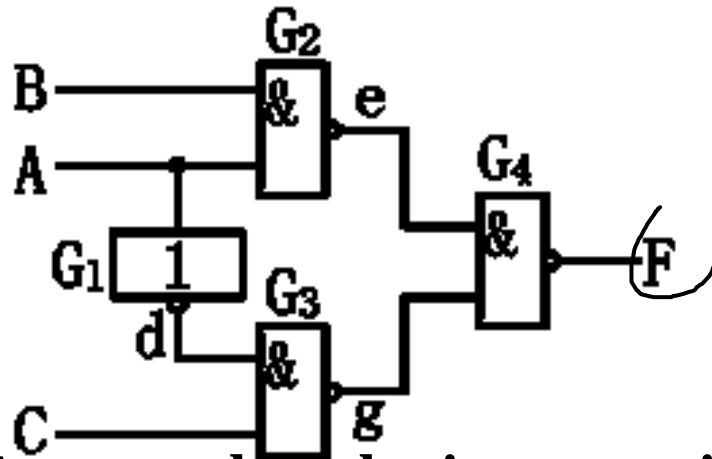
**Non-critical Race** – A race that does not produce erroneous output is called non-critical race.

**Critical Race** – A race that results in erroneous output is called critical race.

**Hazard:** The error output signals derived from the races.

**Note that!** Critical race in combinational circuits are a transient phenomenon, which manifests itself in the generation of unwanted spikes at the output, temporarily breaking the normal logic. Once the transient process is over, the normal logic can be restored.

For example, the following figure shows a combination circuit consisting of a NAND gate with three input variables and one output function.



According to the logic circuit diagram, the output function expression can be written as

$$F = \overline{AB} \cdot \overline{AC} = \overline{AB + AC}$$

$$BC = 11 \quad = A + \bar{A}$$

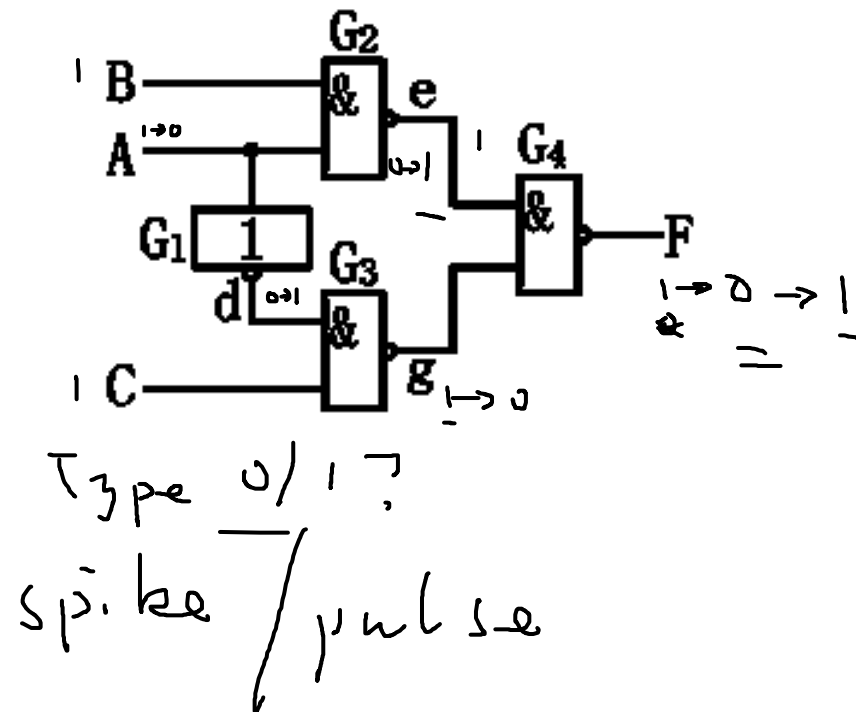
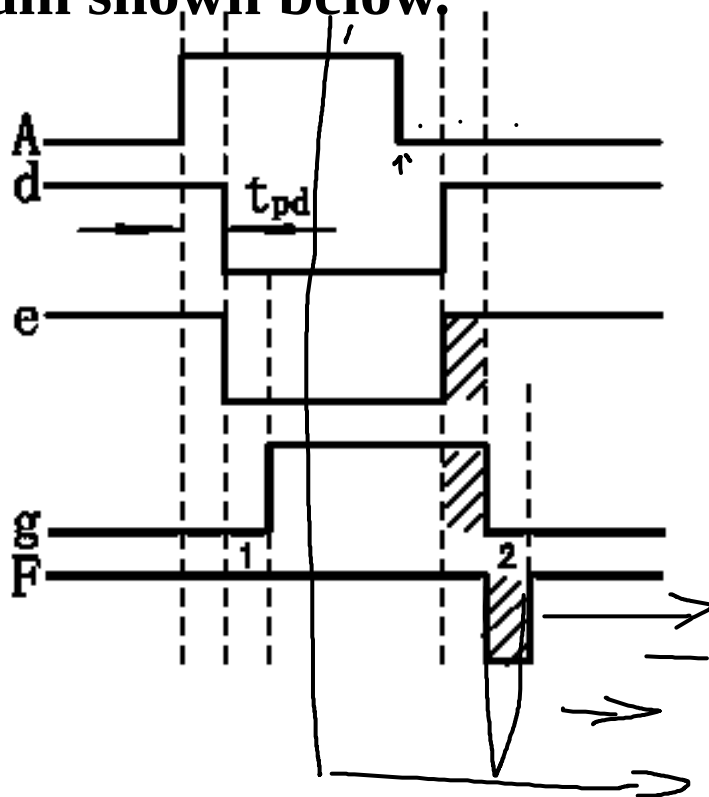
Assume that the input variable  $B = C = 1$ , the value of B, C is substituted into the above function expression, we obtain:

$$F = A + \bar{A}$$

Based on the theorem,  $F = A + \bar{A}$  should be 1, i.e., when  $B=C=1$ , no matter what A is, the output of F should be always 1, cannot be other values.

When considering the time delay in the circuit, what is the actual input and output relationship of the circuit?

Assume that the delay time for each gate is  $t_{pd}$ , the actual input and output relations can be illustrated by the timing diagram shown below.



## 4. 4. 2 Hazard categories

Hazards in combinational circuits can be classified into static hazard and dynamic hazard.

**Static hazard:** If there is a change in the input and the output should not change, a short-term error at the output is called static hazard.

**Dynamic hazard:** If the input changes and the output should change, the output produces a short-term error during the change, which is called a dynamic hazard.

According to the polarity of the wrong output pulse signal can be divided into type "0" of hazard and type "1" of hazard.

**Type "0" hazard:** Error output signal is negative pulse.

**Type "1" hazard:** The error output signal is a positive pulse.

## 4. 4. Hazard determination

For judging whether a circuit can produce hazard, we have algebraic and Karnaugh map methods

### Algebraic method:

● Check whether there is a variable with a race condition in the function expression, that is, whether a certain variable  $X$  is present in the function expression in the forms of an original variable and an complement variable.

● If there is a variable  $X$  with a race condition, other variables in the functional formula are eliminated and check whether the function expression becomes a form of  $X \cdot \bar{X}$  or  $X + \bar{X}$ .  
If true, it indicates that the corresponding logic circuit can produce hazard.



**Example: The logic function expression that is known to describe a combinational circuit is**

$$F = \overline{A}\overline{C} + \overline{A}B + AC$$

Try to determine if the logic circuit may have hazard. It can be seen from the expression that the variables A and C have race conditions. Therefore, these two variables should be analyzed separately. First look at the variable A. To do this, substitute the various combinations of B and C into the function expression. The following results are obtained:

|   |       |                        |                            |                    |
|---|-------|------------------------|----------------------------|--------------------|
| { | BC=00 | $F = \overline{A}$     | AB<br>00<br>01<br>10<br>11 | $F = \overline{C}$ |
|   | BC=01 | $F = A$                |                            | $F = C$            |
|   | BC=10 | $F = \overline{A}$     |                            |                    |
|   | BC=11 | $F = A + \overline{A}$ |                            | ←                  |

It can be seen that when B=C=1, the change of A may cause hazard to the circuit. Similarly, by substituting each value combination of A and B into a function expression, it can be judged from the substitution result that there is no hazard when the variable C changes.



**When describing the logic function of the circuit as an AND-OR expression, it is more intuitive and convenient to use the Karnaugh map to determine the hazard signature than the algebraic method.**

**Karnaugh map method: Make a function Karnaugh map, and draw the K rectangle corresponding to each AND term in the function expression. If there is a "tangential" relationship between two K rectangles (EPI), that is, there are adjacent Minterms between the two K rectangle that are not contained by the same K rectangle, the circuit may produce hazard.**

**For example: if** a logic circuit's function expression **is**  
**Please determine whether it has hazard:**  $F = \overline{A}D + \overline{A}C + A\overline{B}\overline{C}$

**Solution:** the K map can be represented as follows:

2

|    |    | AB |    |    |    |
|----|----|----|----|----|----|
|    |    | 00 | 01 | 11 | 10 |
| CD | 00 | 0  | 0  | 1  | 0  |
|    | 01 | 1  | 1  | 1  | 0  |
|    | 11 | 1  | 1  | 0  | 0  |
|    | 10 | 1  | 1  | 0  | 0  |

1

Based on the K map, Rectangle 1 and Rectangle 2 have the adjacent Minterms:  $m_5$  and  $m_{13}$ ; in addition,  $m_5$  and  $m_{13}$  cannot be covered a same rectangle, So the two K rectangle are "tangential". This indicates that the corresponding circuit may have the hazard.

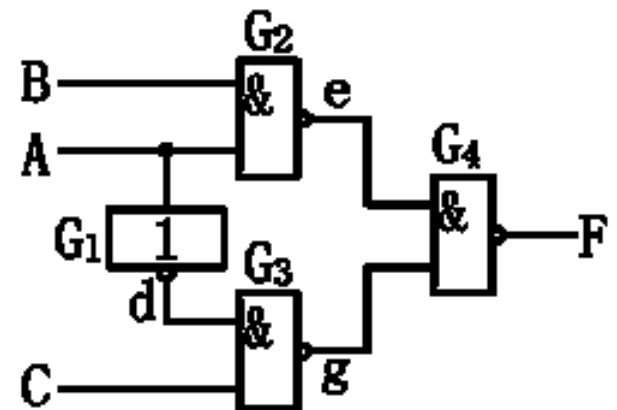
The results can be verified by algebraic method, assuming that  $B=D=1, C=0$ , after the substitution of the function expression  $F$ ,  $F = A + \overline{A}$  can be obtained, it can be seen that the corresponding circuit may be caused by the change of  $A$ .

**Example: Possible hazards in the circuit shown in the right figure.**

**The output function expression for the circuit shown in the figure is**

$$F = AB + \bar{A}C$$

**When  $B = C = 1$  in this circuit, the change in  $A$  may cause the output to produce a "0" type signature.**



#### 4. 4. 4 Elimination of hazard